how easy it is to find incorrect behavior and add new behavior. One reason why it is sometimes hard to pin down the benefit of testing is that good testing often just feels like you're doing a really good job programming.

Of course, it doesn't always work out that way. The tests might have bugs. They might be slow. Environmental issues may mean things that work in a test environment won't work in a development environment. Code changes will break tests. Adding tests to existing code is a pain. As with any other programming tool, there are a lot of ways to cause yourself pain with testing.

Testing First Drives Design

Test-driven development, or TDD, is the counterintuitive idea that developers will improve both the design and the accuracy of their code by writing the tests before they write the code. When adding new logic to a program, the TDD process starts by writing an automated test that describes the behavior of code that does not yet exist. In a strict TDD process, new logic is added to the program only in response to a failing test.

Writing tests before code, rather than after, allows your tests to help guide the design of your code in small, incremental steps. Over time this creates a well-factored codebase that is easy to change.

Success with test-driven development starts with trusting the process. The classic process goes like this:

1. Create a test. The test should be short and test for one thing in your code. The test should run automatically.
2. Make sure the test fails. Verifying the test failure before you write code helps ensure that the test really does what you expect.
3. Write the simplest code that could possibly make the test pass. Don't worry about good code yet. Don't look ahead. Sometimes, write just enough code to clear the current error.
4. After the test passes, refactor to improve the code. Clean up duplication. Optimize. Create new abstractions. Refactoring is a key part of design, so don't skip this. Remember to run the tests again to make sure you haven't changed any behavior.

Repeat until done. This will, in theory, ensure that your code is always as simple as possible and is always completely covered by tests. I'll spend most of this book talking about how to best manage this process using the tools of the Rails ecosystem and solving the kinds of problems that you get in a